

TopGrep Tech Software Developer (Go, Rust) - What to Learn

Overview

This role explicitly lets you pick EITHER Go OR Rust (JD says "using Go or Rust") — good news, since it means you only need to get conversational in one, not both. Your MERN stack + DSA + REST API/microservices experience is a strong genuine match for everything else; the language itself is the only real gap. Go is the faster ramp of the two (simpler syntax, closer to what you already know); Rust is a bigger investment (ownership/borrowing model with no direct analog in your current languages). Recommendation: lead with Go unless you have a specific reason to prefer Rust.

MUST learn

Go basics (recommended primary choice) — syntax, structs, goroutines/channels

- What it is: a simple, statically-typed, compiled language built around fast compilation and first-class concurrency (goroutines = lightweight threads, channels = typed pipes for communication between them).
- Why the JD wants it: it's the JD's first-listed language, and Go is a very common choice for exactly this kind of backend/microservices role (fast startup, small binaries, simple deployment).
- The 20%: write one small REST API in Go using the standard library (`net/http`) or a lightweight router (Gin/Chi) — one GET/POST pair returning JSON. Then write one tiny goroutine+channel example (a few goroutines doing work, sending results back over a channel). That pair covers the core interview vocabulary: structs (Go's version of a class/object shape), goroutines, channels, and basic error handling (Go's explicit `if err != nil` pattern, very different from JS try/catch).
- Bridge: Express's `app.get('/route', (req, res) => ...)` maps to Go's `http.HandleFunc("/route", func(w, r) ...)` — same request/response shape, just typed and more explicit. Goroutines are Go's answer to the async work you already reason about in Node (event loop) and Inngest (Vibe's background jobs).

OR Rust basics (alternative choice) — ownership, borrowing, structs/enums

- What it is: a systems language with memory safety guaranteed at compile time via its ownership model (every value has exactly one owner; borrowing lets you reference it temporarily without owning it) — no garbage collector, no null pointers, no data races, enforced by the compiler.
- Why the JD wants it: listed as an equal alternative to Go — likely for performance-critical services.
- The 20%: only pursue this if you'd rather specialize in Rust than Go — it's a genuinely bigger lift (the ownership/borrowing model has no direct analog in JS/Python/Java). If you do: write one tiny program that passes a `String` to a function, see the "value moved" compiler error, then fix it with a borrow (`&str`) — that single exercise is the crux of what makes Rust different from every language you know.
- Bridge: none direct — this is the one concept in either language that doesn't map cleanly onto JS/TS's garbage-collected model. Budget more time here than for Go if you choose this path.

System design basics

- What it is: high-level tradeoffs in designing a backend system — load balancing, caching layers, database choice (SQL vs. NoSQL), horizontal vs. vertical scaling, API design, and where microservices boundaries should sit.
- Why the JD wants it: named explicitly plus "contribute to system design and architecture discussions" as a responsibility.
- The 20%: you already have real design decisions to point to — Echo's multi-tenant, 7-workspace monorepo architecture and Vibe's containerized sandbox execution model are genuine system-design tradeoffs you made. Practice articulating WHY (multi-tenancy for a B2B SaaS, sandboxing for

untrusted code) rather than just WHAT you built — that's what "system design discussion" actually tests at this level.

Not worth deep prep

MERN stack, REST APIs, microservices concepts, DSA, Git, PostgreSQL/MongoDB — all already genuine, verified experience from your Clink AI work and the Echo/Vibe projects. No new prep needed there; just make sure they're visible in interviews (they already are on this resume variant).